OpenWAM：世界—动作模型的模块化预训练

模型架构、联合建模、训练推理与实验设置

作者：Yang kunpeng

原论文：OpenWAM: An Open, Modular Exploration Towards Systematic World–Action Model Pretraining　论文链接

开源代码：OpenWAM-Official/OpenWAM　代码链接

目录

符号约定 **iii**

第一章 整体架构 **1**

1.1 双流模型 1

1.2 多模态表示 4

第二章 核心方法 **6**

2.1 条件调制 6

2.2 联合注意力 9

2.3 联合训练 14

2.4 联合推理 17

第三章 实验设置 **20**

3.1 视频主干实验 20

3.2 视觉表征实验 21

3.3 网络架构实验 21

3.4 注意力掩码实验 22

3.5 去噪调度实验 22

3.6 跨域预训练实验 23

3.7 仿真评估 24

3.8 真机评估 25

符号约定

符号	含义
B, S_v, S_a	Batch 大小、视频与动作 Token 数目；动作块含 H 步时通常 $S_a = H$。
D_v, D_a	视频与动作残差流宽度，分别为 3072 和 1024。
d_h, D_{attn}	每头注意力维度及总注意力宽度，$d_h = 128$，$D_{\text{attn}} = 24 \times 128$。
Q, K, V, M_{add}	联合注意力的 Query、Key、Value 和加性可见性掩码；代码中用布尔掩码表达同一可见性。
$\boldsymbol{z}, \boldsymbol{a}$	干净视频潜在表示与统一机器人动作块。
$\boldsymbol{\epsilon}_v, \boldsymbol{\epsilon}_a$	视频和动作噪声；下标 v, a 分别对应两条流。
t_v, t_a, σ	视频与动作 Flow 时间，以及代码中的噪声比例；此处 $t = 1 - \sigma$。
$\boldsymbol{m}, \boldsymbol{q}$	动作有效维度掩码与机器人本体状态，后者使用统一的 80 维表示。
s, c, g	AdaLN 的 Shift、Scale、Gate 向量；下标 msa、mlp 表示子模块。
$\mathcal{L}_v, \mathcal{L}_a$	视频速度损失、动作速度损失；动作损失受有效性掩码约束。
θ_v, θ_a	世界流和动作流的可训练参数集合。

第 1 章

整体架构

1.1 双流模型

世界—动作模型（World–Action Model，WAM）同时学习环境未来状态与机器人动作。给定当前观察、语言指令和机器人状态，模型一方面预测未来视频表示，另一方面预测一段连续动作。视频预测提供关于物体、场景和交互动态的监督；动作预测则要求模型把这些动态知识转化为可执行的控制量。OpenWAM 的核心不是把一个视频生成器简单接到动作网络前面，而是让世界流与动作流在 Transformer 的注意力计算中建立显式的信息通路。[1]

OpenWAM 将模型设计拆成三个可组合部分：视觉编码器 $\mathcal{E}$、流式骨干网络集合 $\mathcal{S}$ 和可见性注意力掩码 $\mathcal{M}$。视觉编码器把观察映射到世界流预测的潜在表示；流式骨干网络分别处理世界、动作以及可选的理解流；注意力掩码规定不同模态内部和模态之间哪些 Token 可以相互读取。一个具体模型可写为

$$\mathcal{A} = C(\mathcal{E}, \mathcal{S}, \mathcal{M}),$$

其中 C 是组合规则。组合规则决定各个骨干网络如何执行 prepare、逐层 block 和 finalize，以及是否在注意力计算处拆分为 pre-attention 与 post-attention 两部分。组合器本身不引入额外可训练参数，学习能力保留在各个流的骨干网络中。

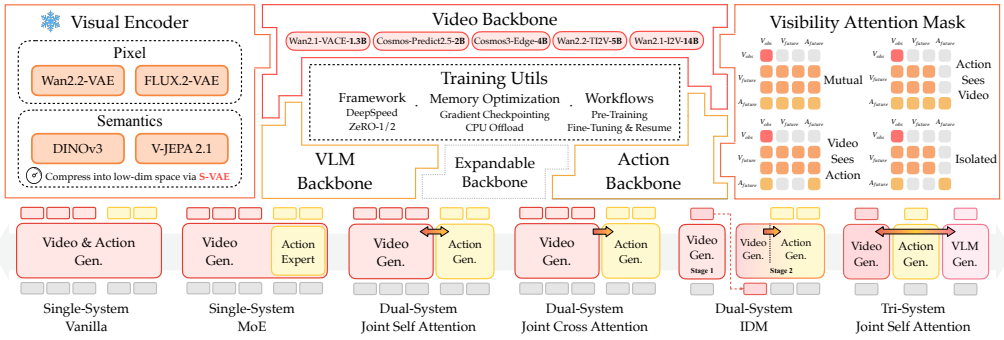


图 1.1: OpenWAM 的模块化模型结构与架构组合方式。

OpenWAM-Infra 统一模型、训练、部署和评估接口；OpenWAM-Study 通过控制变量实验比较架构、信息流和预训练数据策略；OpenWAM-α 则把筛选出的设计组合成一个大规模预训练模型。最终模型的关键机制包括 Wan2.2 视频流、独立 ActionDiT、联合自注意力、可见性掩码和联合 Flow Matching。

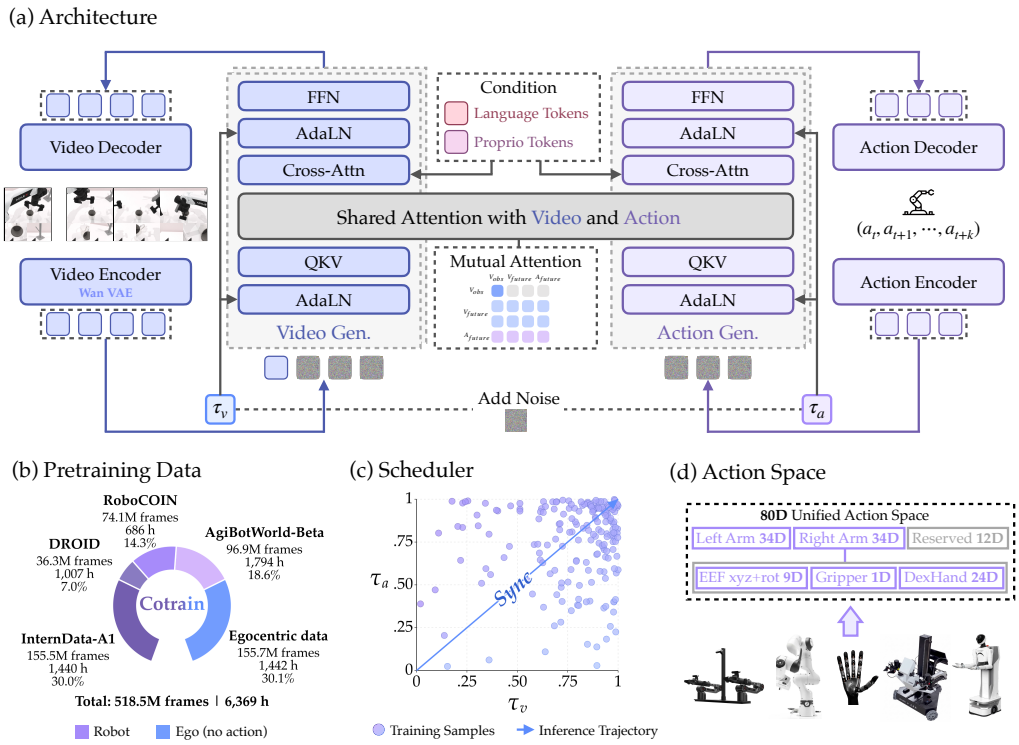


图 1.2: OpenWAM-α 的双流结构、预训练数据、联合噪声平面与统一动作空间。

OpenWAM-α 由两个具有独立残差流的 Transformer 组成：World Stream 使用预训练 Wan2.2-TI2V-5B DiT，Action Stream 使用约 1B 参数的 ActionDiT。Wan2.2-VAE 负责把视频压缩到潜在空间；umT5 将语言指令编码为上下文；机器人本体状态经过线性投影成为额外上下文 Token。两个流各自保留输入投影、归一化、前馈网络和输出头，但在对应 Transformer 层中把 Q、K、V 合并，执行一次混合自注意力。

在最终配置中，两个骨干网络有 30 个对应层。视频残差维度为 3072，动作残差维度为 1024；二者的残差维度不要求相同，但每层都把 Q、K、V 投影到相同的注意力空间，即 24 个注意力头、每头 128 维，总计 3072 维。这个约束使两个流能够在同一次 scaled dot-product attention 中计算，同时仍保留各自独立的残差表示。

视频模型及其输入表示。

Wan2.2 是视频生成基础模型系列。OpenWAM-α 使用其中的 TI2V-5B 作为世界流骨干。它不是 Wan2.2-A14B 的双专家 MoE 版本：TI2V-5B 是 Dense DiT；A14B 系列采用高噪声与低噪声专家按去噪阶段切换的 MoE 设计。两者不能混为一谈。OpenWAM 使用 TI2V-5B 的原因，是在继承视频生成先验的同时控制训练和部署成本。

视频首先由冻结的 Wan2.2-VAE 编码。其时间编码具有因果结构：第一帧单独编码，之后的帧按每组四帧编码。对于论文采用的 $T = 4k + 1$ 帧窗口（k 为非负整数），潜在视频帧数为

$$T' = 1 + \frac{T - 1}{4}.$$

例如输入 $T = 33$ 帧时有 $T\prime = 9$ 个潜在时间位置。第一帧潜在表示保留为当前观察的干净条件，后续潜在帧作为未来世界状态进行去噪。这里的等式依赖输入帧数满足上述分组条件，不能不加说明地用于任意长度窗口。视频潜在经过 $(1, 2, 2)$ 的 patch embedding 后展平成 Token 序列，Token 宽度为 3072，并使用覆盖帧、高度和宽度三个轴的 3D RoPE。

动作序列由 H 个时间步组成，每个时间步的动作向量经过线性投影形成一个 Action Token。OpenWAM-α 的动作流隐藏宽度为 1024，动作 Token 使用沿动作块索引的一维 RoPE。语言指令由冻结的 umT5 编码为 4096 维上下文；机器人本体状态 $\boldsymbol{q} \in \mathbb{R}^{80}$ 经线性映射后追加为一个上下文 Token。两个流分别通过自己的上下文交叉注意力读取语言和本体状态。

统一本体动作空间的目的是让不同机器人平台的动作维度落在同一套固定语义坐标中。80 维向量由两个镜像的 34 维手臂槽位和 12 个预留槽位组成。每个 34 维手臂槽位包括末端位置 3 维、6D 旋转表示 6 维、夹爪 1 维和灵巧手 24 维；最后 12 维用于移动底盘等本体专属通道。各数据集提供从原生动作维度到统一槽位的索引映射 π：

$$\boldsymbol{u} = \mathrm{Scatter}_{\pi}(\mathrm{Norm}(\boldsymbol{a})), \qquad \boldsymbol{a} = \mathrm{Norm}^{-1}(\mathrm{Gather}_{\pi}(\boldsymbol{u})).$$

归一化在 scatter 之前执行，反归一化在 gather 之后执行。本体状态使用同一映射进入统一空间。有效性掩码 $\boldsymbol{m}$ 标记当前本体实际提供的维度；未映射槽位不参与动作监督。

对象	表示与处理	关键维度
视频观察	Wan2.2-VAE 因果编码，随后进行时空 patch embedding	潜在 Token 宽度 3072
动作序列	每个动作时间步线性映射为一个 Action Token	动作流宽度 1024
语言指令	冻结 umT5 编码	上下文宽度 4096
本体状态	80-D 统一状态经线性投影成为上下文 Token	原始宽度 80
联合注意力	两个流独立投影到共享多头注意力空间	$24 \times 128 = 3072$

1.2 多模态表示

OpenWAM-α 的预训练数据包含五个来源、三种数据类型：第一视角人类视频、真实机器人轨迹和仿真机器人轨迹。人类数据提供广泛的场景与交互视觉覆盖，但没有机器人动作标签；真实机器人数据提供可执行动作监督；仿真数据补充单臂与双臂操作技能及环境变化。论文报告原始数据池约 13.339 亿帧、约 14,348 小时，经清洗和按来源抽样后，实际训练集为 518.5M 帧、6,369 小时。

数据来源	类型	训练帧数	小时数	训练占比
Egocentric data	人类视频	155.7M	1,442	30.1%
AgiBotWorld-Beta	真实机器人	96.9M	1,794	18.6%
RoboCOIN	真实机器人	74.1M	686	14.3%
DROID	真实机器人	36.3M	1,007	7.0%
InternData-A1	仿真机器人	155.5M	1,440	30.0%
总计		518.5M	6,369	100%

不同数据源的输入与监督可用性见表 1.1。人类第一视角数据含 71.6K 段长视频（约 0.25–6 分钟），覆盖 3,006 类日常任务；真实机器人数据覆盖 17 个物理平台。

表 1.1: 不同训练来源的输入模态与监督通道。

样本类型	可用输入	有效监督
人类第一视角视频	视频、语言；无有效机器人状态与动作标签	视频预测；动作有效掩码关闭
真实机器人轨迹	视频、语言、机器人状态及动作	视频预测与有效动作维度预测
仿真机器人轨迹	视频与仿真状态、动作等可用模态	视频预测与有效动作维度预测

一个训练样本可表示为

$$(\ell,\ \boldsymbol{o}_{1:T},\ \boldsymbol{a}_{1:H},\ \boldsymbol{q},\ \boldsymbol{m}),$$

其中 ℓ 是语言指令，$\boldsymbol{o}_{1:T}$ 是视频窗口，$\boldsymbol{a}_{1:H}$ 是动作块，$\boldsymbol{q}$ 是可选本体状态，$\boldsymbol{m}$ 是动作各维的有效性掩码。输入准备阶段将视频编码为潜在 $\boldsymbol{z}$，将语言及可用的本体状态组织为上下文 $\boldsymbol{c}$。视频和动作分别进入各自的噪声采样路径；动作有效性掩码决定哪些动作维度参与损失。

数据清洗分为视觉层与机器人信号层。所有来源先剔除无法解码、冻结或重复帧、纯色帧、过曝或欠曝、模糊和突变片段；人类视频还会移除双手离开视野以及语言标注无效的片段。机器人数据进一步检查动作与末端状态的一致性、视频与信号时间对齐、静止片段、状态突变及视频冻结问题。人类视频没有动作监督，并不意味着把人类手部轨迹伪造成机器人动作；它在训练目标中只承担世界预测监督。

动作块的 H 个时间步对应 $S_a = H$ 个 Action Tokens；视频 Token 长度记为 S_v。两种 Token 在残差流中分别保留 $D_v = 3072$ 和 $D_a = 1024$ 的宽度，仅在第 2.2 节的共同注意力空间中对齐，不直接拼接不同宽度的残差表示。

第 2 章

核心方法

2.1 条件调制

AdaLN 的作用是把去噪时间条件注入 Transformer Block。它不直接生成动作，也不替代注意力计算；它根据当前流的噪声状态生成调制参数，改变注意力和前馈网络接收的特征，并通过 Gate 控制子模块输出对残差流的贡献。

以 ActionDiT 为例，动作去噪时间 t_a 首先经过正弦时间编码和 MLP：

$$e_a = \text{TimeEmbedding}(t_a).$$

随后，时间调制网络把 e_a 投影成每层所需的参数：

$$T_a = \text{TimeModulation}(e_a), \qquad T_a \in \mathbb{R}^{B\times 6\times D_a}.$$

这里 B 是批大小，D_a 是动作流隐藏维度。六组向量分别是注意力分支和 FFN 分支的 Shift、Scale 与 Gate。官方实现中的 `TimestepEmbedding` 使用正弦嵌入和两层 MLP；`TimestepModulation` 使用 SiLU 与线性层，将最后一维展开为六组调制参数。

每个 ActionDiT Block 还拥有自己的可学习调制偏置 P_l。当前层将偏置与时间调制相加，再切分为六组：

$$[s_{\text{msa}}, c_{\text{msa}}, g_{\text{msa}}, s_{\text{mlp}}, c_{\text{mlp}}, g_{\text{mlp}}] = \text{Chunk}_6(P_l + T_a).$$

为避免把符号与具体实现顺序混淆，以下使用 s 表示 Shift、c 表示 Scale、g 表示 Gate。注意力输入为

$$h_{\text{msa}} = \text{LN}(x_a) \odot (1 + c_{\text{msa}}) + s_{\text{msa}},$$

注意力结果通过 Gate 加入残差：

$$x'_a = x_a + g_{\text{msa}} \odot \text{SA}(h_{\text{msa}}).$$

接着，动作流读取语言和本体上下文，再进入 FFN：

$$h_{\text{mlp}} = \text{LN}(x'_a) \odot (1 + c_{\text{mlp}}) + s_{\text{mlp}},$$

$$x_a^{\text{next}} = x'_a + g_{\text{mlp}} \odot \text{FFN}(h_{\text{mlp}}).$$

计算次序见表 2.1。文本/本体条件的 Cross-Attention 不采用上述时间 Gate；时间调制作用于联合 Self-Attention 和 FFN。

表 2.1: ActionDiT 每层条件调制的计算顺序；注意力输出已包含对应的输出投影。

步骤	计算关系	作用
时间调制	$T_a = \text{TimeModulation}(e_a)$，与层参数 P_l 相加后六路拆分	生成 Shift、Scale、Gate
注意力输入	$\text{LN}(x_a) \odot (1 + c_{\text{msa}}) + s_{\text{msa}}$	进入 QKV 计算
残差更新	$x'_a = x_a + g_{\text{msa}} \odot \text{SA}(h_{\text{msa}})$	通过 Gate 控制更新
FFN 更新	$x_a^{\text{next}} = x'_a + g_{\text{mlp}} \odot \text{FFN}(h_{\text{mlp}})$	完成该层变换

联合自注意力路径并不直接调用该 Block 的整体 `forward`，而是由 MoT 驱动器分别调用动作流的 pre-attention 与 post-attention 接口。前半部分完成 AdaLN、Q/K/V 投影和一维 RoPE；联合注意力返回动作侧输出后，后半部分执行注意力 Gate、语言/本体上下文 Cross-Attention、FFN 调制及第二个 Gate。

以下官方计算节选分别来自动作块的前后两个接口，展示调制量对残差更新的作用。两段代码不连续，不能独立运行。[1]

[1]来源：OpenWAM 官方仓库，`openwam/model/action_backbone/separate_action_dit.py`；Apache-2.0 许可。

Listing 2.1: ActionDiT 的 AdaLN 与 Gate 计算（官方源码非连续节选）。

```
chunks = (block.modulation.to(
    dtype=payload.t_mod.dtype,
    device=payload.t_mod.device
) + payload.t_mod).chunk(6, dim=1)
shift_msa, scale_msa, gate_msa, shift_mlp, scale_mlp, gate_mlp = chunks
attn_input = block.self_attn_norm(residual_x) * (1 + scale_msa) + shift_msa
# ...
x = block.gate(residual_x, gate_msa, block.self_attn.o(attn_out))
mlp_input = block.ffn_norm(x) * (1 + scale_mlp) + shift_mlp
x = block.gate(x, gate_mlp, block.ffn(mlp_input))
```

adaLN-Zero 与 OpenWAM ActionDiT 的初始化方式不同。原始 DiT 的 adaLN-Zero 将调制网络最后一层零初始化，使初始 Shift、Scale 和 Gate 为零，Block 初始接近恒等映射。OpenWAM 当前 ActionDiT 的代码则把每个 Block 的 `modulation` 参数初始化为 $\mathcal{N}(0, 1/D_a)$，并通过常规 `Linear` 生成时间调制；这不是严格的 adaLN-Zero 初始化。因此，解释 OpenWAM 时不能直接断言 ActionDiT 的 Gate 初始恒为零。Gate 仍然是可学习的条件调制量，但其初始行为应以实际参数初始化为准。

ActionDiT 的时间调制数据流在 `prepare_state` 中完成：噪声动作先经 `action_encoder` 映射，动作时间经 `time_embedding` 和 `time_projection` 生成 `t_mod`，之后把动作隐藏状态、调制参数、RoPE 频率和上下文装入状态对象，供逐层联合注意力驱动器使用。

公式中的 g_{msa} 是特征维度上的条件 Gate 向量，而不是一个必须位于 $[0, 1]$ 的概率值。假设注意力分支的输出为 $F(x)$，则残差形式 $y = x + g \odot F(x)$ 对 Gate 的局部导数为

$$\frac{\partial \mathcal{L}}{\partial g} = \frac{\partial \mathcal{L}}{\partial y} \odot F(x).$$

若将分支输出与参数展平为向量，令 J_{F,θ_F} 为 $F(x;\theta_F)$ 对参数的雅可比矩阵，则

$$\nabla_{\theta_F} \mathcal{L} = J_{F,\theta_F}^{\mathsf{T}} \big(\nabla_y \mathcal{L} \odot g \big).$$

该式使用向量形式的链式法则，避免把向量梯度与雅可比矩阵误作普通标量相乘；条件调制参数若也依赖其他可训练权重，仍应计算其对应的链式梯度。这两个导数显示：即使某种初始化令 $g = 0$，Gate 本身仍可能获得非零梯度，而经该残差项传

给分支权重的梯度暂时为零。这是解释性推导，不意味着 OpenWAM ActionDiT 采用零 Gate 初始化。

2.2 联合注意力

Joint Attention 的关键是：两个流分别从自己的隐藏状态生成 Q、K、V，但把两个模态的序列维拼接起来，只执行一次混合注意力。令视频流和动作流的序列长度分别为 S_v 与 S_a，则

$$Q = [Q_v; Q_a], \qquad K = [K_v; K_a], \qquad V = [V_v; V_a].$$

在单个注意力头内，令 $Q_v \in \mathbb{R}^{S_v \times d_h}$、$Q_a \in \mathbb{R}^{S_a \times d_h}$，对应的 K, V 形状相同。拼接后 $Q, K, V \in \mathbb{R}^{(S_v+S_a)\times d_h}$。混合注意力为

$$O = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_h}} + M_{\text{add}}\right) V,$$

其中 d_h 是每头维度，M_{add} 是加性可见性掩码：允许的 Query–Key 配对赋值为 0，禁止的配对赋值为 $-\infty$。上述矩阵的形状为 $(S_v + S_a) \times (S_v + S_a)$；实现时还需考虑批次与注意力头的广播。两模态的四个 Query–Key 区域明确为

$$QK^{\mathsf{T}} = \begin{bmatrix} Q_v K_v^{\mathsf{T}} & Q_v K_a^{\mathsf{T}} \\ Q_a K_v^{\mathsf{T}} & Q_a K_a^{\mathsf{T}} \end{bmatrix}.$$

右上区域描述世界流读取动作，左下区域描述动作流读取世界；它们在共同 Softmax 中与各自模态内的 Key 一起竞争权重。得到的结果按原始序列长度拆分：

$$O = [O_v; O_a], \qquad O_v \in \mathbb{R}^{B\times S_v \times D_{\text{attn}}}, \quad O_a \in \mathbb{R}^{B\times S_a \times D_{\text{attn}}}.$$

两个流分别把自己的注意力输出送回对应 Block 的 post-attention 部分，再经过各自的输出投影、上下文交叉注意力和 FFN。

两个流的残差维度可以不同。Joint Attention 只要求它们的注意力头数和每头维度匹配，从而能把 Q、K、V 放进同一个注意力空间。OpenWAM-α 使用 24 个头、每头 128 维；视频残差维度为 3072，动作残差维度为 1024。每个流自己的投影层负责把残差特征映射到共同的 24×128 注意力空间。

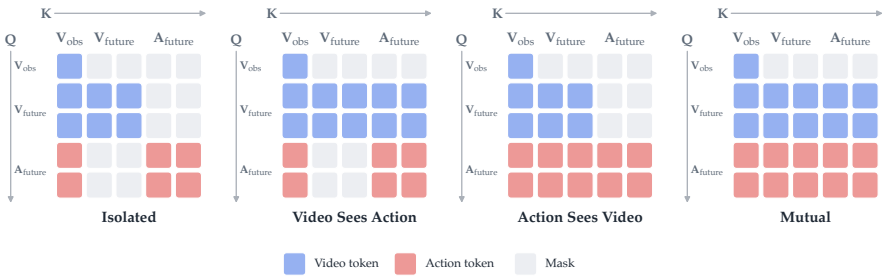


图 2.1: 跨模态可见性策略及其信息流方向。

官方 `DualSystemMoTDriver` 的逐层执行顺序是：先分别调用视频和动作骨干的 pre-attention 接口；沿序列维拼接 Q、K、V；用 `scaled_dot_product_attention` 计算混合注意力；按 S_v, S_a 拆分输出；最后分别调用两个骨干的 post-attention 接口。这个驱动器本身不含可训练参数，参数仍归属各自的骨干网络。

联合注意力驱动器把两个流的 Q、K、V 沿序列维拼接，并在一次注意力调用后按原始长度拆分。下列官方节选采用不启用内部 Attention Checkpoint 的执行分支。[2]

Listing 2.2: 两个流的 QKV 拼接与联合输出拆分（官方源码节选）。

```
q_v, k_v, v_v, vpost = vb.pre_attn_at_layer(layer_id, vstate)
q_a, k_a, v_a, apost = ab.pre_attn_at_layer(layer_id, astate)
s_video = q_v.shape[1]
s_action = q_a.shape[1]
q_cat = torch.cat([q_v, q_a], dim=1)
k_cat = torch.cat([k_v, k_a], dim=1)
v_cat = torch.cat([v_v, v_a], dim=1)
mixed = self._mixed_attention(q_cat, k_cat, v_cat, attn_mask)
attn_v, attn_a = mixed.split([s_video, s_action], dim=1)
```

混合注意力内核把 (B, S, Hd_h) 重排为 (B, H, S, d_h)，执行 PyTorch SDPA，再还原到原有布局。

[2]来源：OpenWAM 官方仓库，`openwam/model/architectures/dual_system/mot_driver.py`；Apache-2.0 许可。

掩码矩阵可分解为四个子块：

$$M_{\text{add}} = \begin{pmatrix} M_{V\leftarrow V} & M_{V\leftarrow A} \\ M_{A\leftarrow V} & M_{A\leftarrow A} \end{pmatrix}.$$

$M_{X\leftarrow Y}$ 表示加性掩码中模态 X 的 Query 对模态 Y 的 Key 所采用的分块；通过取 0 或 $-\infty$ 规定是否可读取。视频内部使用 first-frame-causal 规则：干净的首帧只读取自身，未来带噪视频帧可以读取首帧以及允许范围内的未来帧；动作内部采用双向可见。跨模态掩码的四种配置见表 2.2。这里“视频”特指未来预测视频 Token，不把干净首帧的条件读取误算为一般跨模态桥接。

表 2.2: 联合自注意力的跨模态可见性；干净首帧 Query 不读取动作。

模式	未来视频读动作	动作读未来视频	动作读首帧
Mutual	允许	允许	允许
Action Sees Video	禁止	允许	允许
Video Sees Action	允许	禁止	允许
Isolated	禁止	禁止	允许

公式使用加性掩码 M_{add}；官方 PyTorch SDPA 实现则传入布尔掩码 M_{bool}：`True` 表示可见，`False` 表示禁止。两种记法表达相同的可见性，但数值不可直接混加。矩阵的行是 Query，列是 Key。OpenWAM-α 采用 Mutual 与 first-frame-causal：干净首帧 Query 仅读取首帧；未来视频和动作互相可见，并且都可以使用当前观测。

以下官方节选保留控制四种跨模态可见性模式的赋值分支。[3]

[3]来源：OpenWAM 官方仓库，`openwam/model/architectures/utils/mask_modes.py`；Apache-2.0 许可。

Listing 2.3: 跨模态可见性矩阵的核心分支（官方源码节选）。

```
s_video = v_end - v_start
ff = min(video_tokens_per_frame, s_video)
if mode in (ACTION_SEES_VIDEO, MUTUAL):
    mask[a_start:a_end, v_start:v_end] = True
else:
    mask[a_start:a_end, v_start:v_end] = False
    mask[a_start:a_end, v_start : v_start + ff] = True
if mode in (MUTUAL, VIDEO_SEES_ACTION):
    mask[v_start:v_end, a_start:a_end] = True
    mask[v_start : v_start + ff, a_start:a_end] = False
else:
    mask[v_start:v_end, a_start:a_end] = False
```

与 Joint Cross-Attention 的区别。Joint Cross-Attention 在动作侧以 Q_a 查询视频侧 K_v, V_v，分数矩阵的尺寸为 $S_a \times S_v$，动作 Query 的 Softmax 只在视频 Key 之间归一化。Joint Self-Attention 的动作 Query 则同时读取视频和动作两类 Key：

$$\mathrm{softmax}\left(\frac{[Q_a K_v^\top,\ Q_a K_a^\top]}{\sqrt{d_h}} + M_a\right).$$

从而两类 Token 在相同的 Softmax 中竞争注意力权重；并且如果开放视频 Query 读取动作 Key 的区域，世界流也将依赖动作流。两个架构不仅在方向上不同，归一化范围和残差更新位置也不相同。对于 Cross-Attention，若对桥接视频特征执行 `detach()`，动作损失不能通过该桥接特征更新视频骨干；不执行 `detach()` 则可能反向传播。这里的 *bridge* 与给文本和 Proprioception 使用的上下文 Cross-Attention 不是同一个模块。

首帧例外与代码语义。在 Mutual 模式下，干净首帧的 Query 不读取带噪未来帧，也不读取 Action Key；未来视频和动作则允许双向交换信息。对应实现还规定：在 `video-sees-action` 和 `isolated` 等禁用一般 Action←Video 桥接的模式下，仍保留 Action Query 对视频首帧的读取权。这是上下文观测通路，不代表动作可以读取全部未来视频。因此四种模式比较的是预测流间的跨模态可见性，不能机械解释为彻底禁止所有视频条件。`scaled_dot_product_attention` 使用布尔 Mask 时，`True` 代表可以关注，和 PyTorch 部分其他注意力接口的 Mask 约定不同。

人类第一视角数据没有机器人动作和本体状态标签，论文定义为仅提供世界预测监督。实现中必须区分表 2.3 的三个掩码，不能因为动作损失为零就推断动作 Token 不参与联合注意力。

表 2.3: 三种掩码控制不同的计算环节，不能相互替代。

掩码	使用位置	决定的事情
Attention Mask	QK 注意力分数矩阵	Query 是否读取指定 Key
Action Mask	动作 Loss 归约	哪些动作维度 / 时间步有有效监督
Proprioception Mask	上下文 Token 与上下文注意力	本体状态 Token 是否允许被读取

Action 缺失。动作损失只在样本具有动作监督时构造动作目标。当整个 Batch 的 `actions` 为 `None` 时，联合架构的 `forward` 只执行视频骨干，不产生动作预测。若 Reader 提供占位动作 Tensor 和全 False 的 `action_mask`，动作损失只对有效位置归约；`prepare_inputs` 生成的 `action_is_pad` 是损失侧掩码，在 `compute_loss` 中不会作为联合注意力掩码传入。因此，*占位动作若仍进入 Mutual 注意力，视频损失仍可能通过动作 Key/Value 产生反向梯度*。论文没有规定所有数据 Reader 的占位方式，因此上述两条分支不能直接归结为人类视频样本统一采取的处理方式。

输入整理以批次首个样本的动作与本体字段决定是否进行张量堆叠，因此同一批次必须使用一致的字段形状约定。混合不同数据来源时，数据读取器需提供兼容的占位张量与逐样本有效性掩码，或者保证同批样本具有相同的字段结构。论文描述的混合预训练并不直接确定每个批次内的实际混合方式。

Proprioception 缺失。 OpenWAM 将本体状态投影为一个上下文 Token，并把对应布尔值追加到 `context_mask`。若样本级 `proprio_mask` 为 False，代码仍会先调用 `proprio_encoder`，再将得到的 Token 乘以零，并把该位置的上下文可见性设为 False。这样，该样本的本体 Token 不会被上下文交叉注意力读取；乘零也使该样本通过这个 Token 对 `proprio_encoder` 权重产生的梯度贡献为零。

4

[4]来源：OpenWAM 官方仓库，`openwam/model/architectures/base.py`；Apache-2.0 许可。

Listing 2.4: 本体状态 Token 的有效性处理（官方源码节选）。

```
sample_mask_f = sample_mask.to(proprio_token.dtype).unsqueeze(-1)
proprio_token = proprio_token * sample_mask_f
updated = dict(pipeline_inputs)
updated["context"] = torch.cat([context, proprio_token], dim=1)
updated["context_mask"] = torch.cat([context_mask, sample_mask], dim=1)
```

当前实现还要求输入张量满足形状契约：若 use_proprioception=True，但 proprio 直接为 None，_append_proprio_context_token 会抛出异常。因此，缺失状态不能在该路径中简单地以 Python None 代替；需要由数据读取路径提供形状匹配的占位状态并给出 False 掩码，或者关闭本体条件分支。论文层面的“完全屏蔽”是监督语义，不能替代代码层面的输入张量要求。

梯度流向。对机器人样本，视频损失更新视频流，动作损失更新动作流；在 mutual 联合注意力下，视频输出依赖动作 Key/Value，动作输出也依赖视频 Key/Value，因此视频损失可以沿动作 Key/Value 路径影响动作流上游特征，动作损失也可以沿视频 Key/Value 路径影响视频流上游特征。对于没有动作 Token 的人类样本，前向只运行视频流，动作分支没有参与该样本的联合注意力。对于带占位动作 Token 但动作损失全无效的样本，动作损失贡献为零；然而如果这些 Token 仍然进入 mutual 注意力，视频损失仍可能沿视频 Query 对动作 Key/Value 的依赖产生梯度。这个结论来自注意力计算图，而不是来自动作损失掩码本身。

本体状态掩码为 False 时，状态 Token 被置零且上下文掩码关闭，因此该样本不会通过本体 Token 对 proprio_encoder 产生有效梯度。其他有效文本 Token 仍然参与上下文交叉注意力。OpenWAM-α 训练时更新视频 DiT、ActionDiT 和本体状态编码器；Wan2.2-VAE 与 umT5 文本编码器被冻结。官方冻结接口同时执行 requires_grad_(False)、设置 eval 状态，并用 torch.no_grad() 包装被冻结子模块的 forward。梯度检查点则不同：它在反向传播时重算前向激活以节省显存，并不等同于 detach，也不切断梯度路径。

2.3 联合训练

一次训练迭代的数据流见表 2.4。prepare_inputs 将原始视频和文本编码为视频潜在及上下文，并整理动作、状态与有效性掩码。VAE 和文本编码器保持冻结，不由本次视频或动作损失更新。

表 2.4: 一次训练迭代中的数据处理、预测和参数更新。

环节	核心计算与监督关系
输入处理	视频编码为 $\boldsymbol{z}$，动作归一化并映射至 80-D；构造语言、本体条件和有效性掩码
独立加噪	分别取 t_v, t_a 和高斯噪声，形成 $\boldsymbol{z}^{t_v}, \boldsymbol{a}^{t_a}$
双流前向	两条残差流分别投影 QKV，在 Mutual Mask 下联合注意力，再各自输出速度
损失计算	视频速度误差、有效动作维度上的速度误差，加权后相加
反向传播	根据实际 Mask、Gate 和计算图更新两个骨干及其他未冻结参数

视频和动作使用独立噪声时间。论文采用 $t = 0$ 表示纯噪声、$t = 1$ 表示干净数据的约定：

$$\boldsymbol{z}^{t_v} = t_v \boldsymbol{z} + (1 - t_v)\boldsymbol{\epsilon}_v, \qquad \boldsymbol{a}^{t_a} = t_a \boldsymbol{a} + (1 - t_a)\boldsymbol{\epsilon}_a,$$

其中 $\boldsymbol{\epsilon}_v$ 和 $\boldsymbol{\epsilon}_a$ 分别从各自形状匹配的标准高斯分布采样。对固定的干净样本和噪声，插值关于时间的导数分别为 $\mathrm{d}\boldsymbol{z}^{t_v}/\mathrm{d}t_v = \boldsymbol{z} - \boldsymbol{\epsilon}_v$、$\mathrm{d}\boldsymbol{a}^{t_a}/\mathrm{d}t_a = \boldsymbol{a} - \boldsymbol{\epsilon}_a$；因此速度目标来自插值路径的方向导数，而非直接预测高斯噪声。两个流的时间独立采样。模型接收两种带噪表示、各自的时间条件和上下文，联合预测两个速度：

$$(\hat{\boldsymbol{v}}_z, \hat{\boldsymbol{v}}_a) = \boldsymbol{v}_\theta(\boldsymbol{z}^{t_v}, \boldsymbol{a}^{t_a}, t_v, t_a, \boldsymbol{c}).$$

联合 Flow Matching 损失为

$$\begin{aligned}\mathcal{L} = \lambda_v \, \mathbb{E}\Big[w(t_v) \, \|\hat{\boldsymbol{v}}_z - (\boldsymbol{z} - \boldsymbol{\epsilon}_v)\|_2^2\Big] \\ + \lambda_a \, \mathbb{E}\Big[w(t_a) \, \|\boldsymbol{m} \odot (\hat{\boldsymbol{v}}_a - (\boldsymbol{a} - \boldsymbol{\epsilon}_a))\|_2^2\Big].\end{aligned}$$

λ_v 和 λ_a 控制两个流的损失权重，$w(\cdot)$ 是时间权重，$\boldsymbol{m}$ 是动作维度有效性掩码。视频首帧是当前观察的干净条件，不属于未来视频预测损失。动作无效维度不贡献动作损失。这里的 $\boldsymbol{m}$ 是动作监督有效性掩码，不能替代 Joint Attention 的 Query–Key 可见性掩码。

论文符号与代码 sigma 的方向。论文用 t 表示从噪声端到数据端递增的时间；官方调度器使用 σ 表示噪声比例，$\sigma = 1$ 为纯噪声，$\sigma = 0$ 为干净数据。因此二者关系是 $t = 1 - \sigma$。代码中的插值写为

$$x_\sigma = (1 - \sigma)x + \sigma\epsilon,$$

对应的训练目标为 $\epsilon - x$。这与论文写法 $x - \epsilon$ 的符号相反，是因为两套时间坐标的方向相反；从 $\sigma = 1$ 向 $\sigma = 0$ 积分时，代码使用负的 $\Delta\sigma$，仍然朝干净数据方向更新。

视频与动作的加噪插值均由损失计算函数实现，速度目标的符号方向随所用时间坐标而变化。

OpenWAM-α 对 t_v 与 t_a 独立采样，因此训练覆盖二维联合噪声平面，而不是只训练 $t_v = t_a$ 的对角线。论文采用 $\lambda_v = \lambda_a = 1$，两个流的 Flow Shift 均为 5；时间采样经 $\rho = 5$ 的 warp 向高噪声区域偏置。模型通过一次双流前向同时预测视频速度与动作速度，再分别计算两项损失并相加。

训练参数与冻结策略。 OpenWAM-α 使用预训练 Wan2.2-TI2V-5B 初始化视频 DiT，ActionDiT 与本体状态编码器也参与端到端更新；Wan2.2-VAE 与 umT5 文本编码器保持冻结。论文报告的大规模预训练使用 16 台计算节点、每台 8 张 NVIDIA H200，共 128 张 GPU，训练约 7 天。每张 GPU 的 batch size 为 24，梯度累积为 0，全局 batch size 为 3,072 clips，每个 epoch 155,862 次优化迭代。优化器为 AdamW，学习率 10^{-4}，余弦调度、5% warmup、权重衰减 0.01，$\beta_1 = 0.9$、$\beta_2 = 0.95$，梯度裁剪范数 1.0，训练精度为 bfloat16，分布式策略为 DeepSpeed ZeRO Stage 2。

配置项	论文中的设置
计算资源	16 节点，128 张 NVIDIA H200
训练时长	约 7 天
全局 Batch Size	3,072 clips（每 GPU 24）
训练迭代	155,862（1 epoch）
优化器	AdamW，学习率 1×10^{-4}
学习率调度	Cosine，5% warmup，最低比例 0.01
精度与分布式	BF16，DeepSpeed ZeRO Stage 2
输入视频	33 帧，stride 4，window stride 1
图像尺寸	384×320，启用多视角输入
Flow Shift	Video / Action 均为 5.0
损失权重	$\lambda_v = 1.0$，$\lambda_a = 1.0$
统一控制空间	80-D action，80-D proprioception

梯度的逐项解释。用 θ_v、θ_a 分别表示两个可训练流的参数，将总损失写成 $\mathcal{L} = \mathcal{L}_v + \mathcal{L}_a$。在 Mutual 联合注意力下，一般存在

$$\frac{\partial \mathcal{L}}{\partial \theta_v} = \frac{\partial \mathcal{L}_v}{\partial \theta_v} + \frac{\partial \mathcal{L}_a}{\partial \theta_v}, \qquad \frac{\partial \mathcal{L}}{\partial \theta_a} = \frac{\partial \mathcal{L}_v}{\partial \theta_a} + \frac{\partial \mathcal{L}_a}{\partial \theta_a}.$$

这两项交叉偏导是否实际非零还受 Mask、是否存在动作 Token、Gate、detach 以及冻结配置约束。人类视频上的 $\mathcal{L}_a = 0$ 不足以推断 $\partial \mathcal{L}_v / \partial \theta_a = 0$；只有当动作分支未参与此样本的计算，或对相应路径有明确屏蔽与断梯度时，才能得出后者为零。该式是计算图的解释性展开，不额外定义论文未提出的训练损失。

2.4 联合推理

推理按表 2.5 的顺序执行。OpenWAM-α 在联合噪声平面使用同步对角路径，即两个流在相同时间进度下共同去噪。论文使用 $N = 10$ 步，时间路径从 $t^0 = 0$ 单调走向 $t^N = 1$，定义为

$$t^i = 1 - f_\rho\left(1 - \frac{i}{N}\right), \qquad f_\rho(s) = \frac{\rho s}{1 + (\rho - 1)s}, \quad \rho = 5.$$

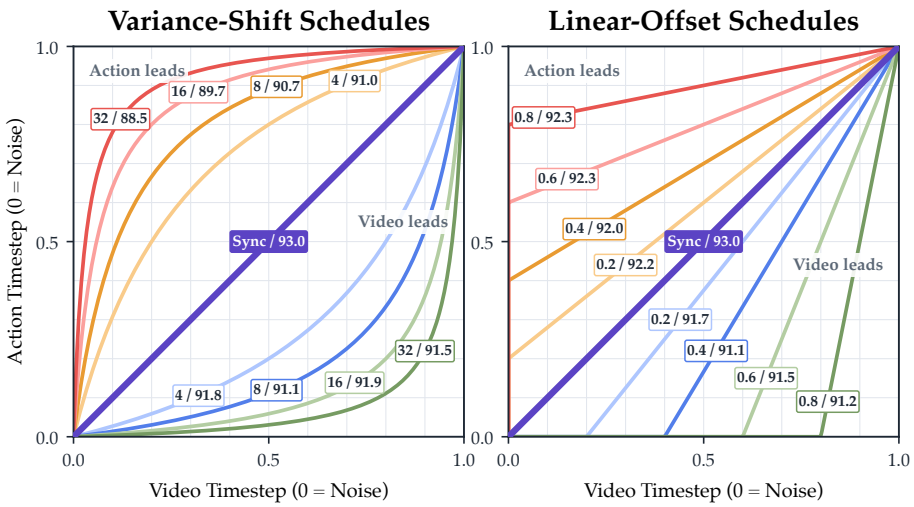


图 2.2: 视频与动作去噪进度在联合噪声平面中的不同路径。

每个去噪步骤执行一次双流联合前向，得到 $\hat{\boldsymbol{v}}_z$ 和 $\hat{\boldsymbol{v}}_a$，再分别按各自的时间增量进行 Euler 更新：

$$\boldsymbol{z}^{t^{i+1}} = \boldsymbol{z}^{t^i} + (t^{i+1} - t^i)\hat{\boldsymbol{v}}_z,$$
$$\boldsymbol{a}^{t^{i+1}} = \boldsymbol{a}^{t^i} + (t^{i+1} - t^i)\hat{\boldsymbol{v}}_a.$$

官方 `generate` 的循环直接使用 schedule 中相邻的两个时间点，分别判断视频流和动作流是否需要更新。联合前向每次同时产生两个速度预测；若某一流处于时间平台期，它仍保留在联合前向上下文中，只是不执行该流的状态更新。视频更新后会重新写入干净首帧；动作更新则调用官方调度器的 Euler `flow_step`。这说明“某一流暂时不去噪”不等于“从联合注意力中删除该流”。

视频第一帧始终固定为当前观察编码得到的干净潜在表示，并在每一步更新后重新钉回该条件帧。其余视频潜在和动作块沿同步时间路径逐步变干净。最终动作块通过统一动作空间的逆映射转换回具体机器人平台的原生动作空间。

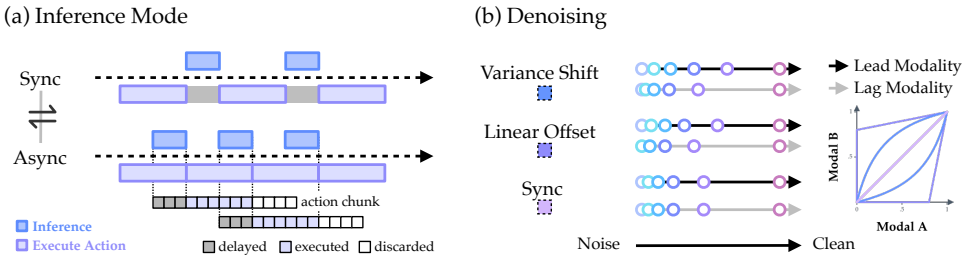


图 2.3: 推理执行模式与视频—动作去噪调度。

表 2.5: 同步联合去噪和机器人动作输出的执行流程。

阶段	执行内容
初始化	编码当前干净首帧，初始化未来视频潜在与动作噪声
联合预测	对同一时间点的两个流执行联合注意力，预测视频和动作速度
迭代积分	按时间增量分别更新两个流，并把视频干净首帧写回
动作输出	读取最终动作块，逆映射并反归一化至机器人原生控制空间
闭环执行	最新观察触发下一次动作块生成，控制器顺序消费动作缓冲区

部署时，动作缓冲区耗尽才根据最新观察同步请求下一动作块；在新块生成前暂停执行。实现可使用固定形状编译、CUDA Graph、速度缓存和提示词缓存；机器人控制路径不需要把未来视频潜在解码为 RGB。

从输出到控制。 ActionDiT 的输出是动作流速度预测，不是直接把隐藏状态当作最终控制量。每个去噪步都更新动作潜变量，最后得到完整动作块。统一动作向

量经 gather 与反归一化后，恢复到当前机器人平台的物理单位。官方生成接口将动作张量移到 CPU 并转成 NumPy 数组；在配置统一动作归一化器时，接口还会执行反归一化。部署端负责使相机、语言与可选本体状态的输入符合训练约定，并将输出还原为平台原生控制量。

第 3 章

实验设置

各实验分别列出测试数据、测试方法、评价指标与对比基线。不同基准的指标和参照方法以论文给出的协议为准，不把不同机器人的任务定义强行合并。[1]

3.1 视频主干实验

在不同视频生成预训练骨干上保持双流联合自注意力架构与下游任务设置，比较可继承的生成先验。

表 3.1: 视频主干实验的实验设置

类别	名称	说明
测试数据	RoboTwin2.0-Full	双臂操作任务；干净训练数据与强随机化训练数据共同构成下游微调集，分别在 Clean 与 Randomized 场景测试。
测试方法	固定双流骨干接口	以预训练视频模型参数初始化 World Stream，ActionDiT 负责动作建模；下游微调与评估协议保持一致。
测试方法	候选视频骨干	Wan2.1-VACE-1.3B、Cosmos-Predict2.5-2B、Wan2.2-TI2V-5B、Wan2.1-I2V-14B。
评价指标	任务成功率	分别度量 Clean、Randomized 条件下完成双臂操作任务的比例，并记录两者均值。
对比基线	四种视频骨干配置	比较生成先验及参数规模的设计差异；各模型还可能在架构与预训练任务上不同，不将差异机械解释为纯规模效应。

3.2 视觉表征实验

保持后续预测器结构，并通过不同视觉编码器及 S-VAE 压缩条件观察视觉表征的可用性。

表 3.2: 视觉表征实验的实验设置

类别	名称	说明
测试数据	RoboTwin2.0-Full	与视频主干实验一致的机器人下游微调及场景测试。
测试方法	重建类编码器	Wan2.2-VAE 与 FLUX.2-VAE；视觉编码器保持冻结。
测试方法	表征类编码器	DINOv3 与 V-JEPA 2.1；非原生视频编码器采用平均池化形成约四倍时域压缩。
测试方法	维度压缩对照	比较直接使用表征特征与通过 S-VAE 压缩到 48-D 的方案；对应视频 DiT 权重随机初始化以隔离表征差异。
评价指标	任务成功率	按 RoboTwin2.0-Full 的 Clean、Randomized 及平均成功率统计。
对比基线	不同表征空间	比较重建编码器、表征编码器、以及表征编码器加 S-VAE 的配置。

3.3 网络架构实验

以统一基准研究动作容量及视频—动作交互方式。

表 3.3: 网络架构实验的实验设置

类别	名称	说明
测试数据	RoboTwin2.0-Full	在 Clean 与 Randomized 条件下评估双臂操作任务。
测试方法	Single-System	Vanilla 与 MoE 两个变体；动作 Token 加入共享视频网络。
测试方法	Dual-System	Joint Self-Attention、Joint Cross-Attention、Detached Cross-Attention 和 IDM。
测试方法	Tri-System	附加理解流的 Joint Self-Attention。
评价指标	任务成功率	分别统计 Clean、Randomized 和平均任务成功率。
对比基线	单流 / 双流 / 三流	比较参数共享、独立 ActionDiT 和增加理解流的不同计算结构。

3.4 注意力掩码实验

将网络固定为双流 Joint Self-Attention，只改变视频和动作的跨模态可见性。

表 3.4: 注意力掩码实验的实验设置

类别	名称	说明
测试数据	RoboTwin2.0-Full	在 Clean、Randomized 场景评估，保持预训练 / 微调条件可对照。
测试数据	RoboTwin2.0-Clean2Random	只用 Clean 数据微调，在 Clean 与 Randomized 两种场景分别测试分布内与分布外情况。
测试方法	四类可见性配置	Isolated、Video Sees Action、Action Sees Video、Mutual；视频内部仍遵守首帧因果规则。
测试方法	训练阶段对照	对照从头微调以及跨域预训练后的相同 Mask 配置，分开考察信息方向与预训练的影响。
评价指标	任务成功率	Clean、Randomized 及平均成功率；按 ID 与 OOD 测试条件报告。
对比基线	相同模型的掩码变体	只改变跨模态 Query–Key 的可见性，控制其他网络与优化条件。

3.5 去噪调度实验

比较同步对角路径与一方提前去噪的推理轨迹。

表 3.5: 去噪调度实验的实验设置

类别	名称	说明
测试数据	RoboTwin2.0-Full	使用与世界—动作信息流消融一致的双臂任务协议。
测试方法	同步去噪	两个流在同一全局进度推进，时间轨迹满足 $t_v = t_a$。
测试方法	Variance Shift	通过 $f_\alpha(s)$ 调整某一流相对全局进度的领先程度。
测试方法	Linear Offset	通过 $h_o(s)$ 延后某一流的进度；区分 Video Lead 与 Action Lead。
评价指标	任务成功率	Clean、Randomized 与平均任务成功率。
对比基线	同步对角路径	将异步 Video Lead、Action Lead 调度与同参数设置下的同步对照。

3.6 跨域预训练实验

在相同预训练时长预算下，区分人类视频覆盖与机器人动作监督的影响。

表 3.6: 跨域预训练实验的实验设置

类别	名称	说明
测试数据	EgoDex 与 RoboCOIN	受控预训练中采用人类第一视角视频与真实机器人轨迹，总预算统一为 600 小时。
测试数据	RoboTwin2.0-Clean2Random	下游只在 Clean 场景微调，分别在 Clean 与 Randomized 场景测试。
测试数据	RoboTwin2.0-Full	下游使用 Clean 与 Randomized 混合数据微调。
测试方法	Robot-only	600 小时机器人轨迹预训练。
测试方法	Ego then Robot	350 小时人类视频加 250 小时机器人数据，采用分阶段训练。
测试方法	Ego + Robot Co-train	350 小时人类视频加 250 小时机器人数据，进行单阶段联合训练。
评价指标	ID/OOD 任务成功率	在相同后续微调流程下分别报告 Clean、Randomized 的成功率。
对比基线	无具身预训练	从头进行下游监督微调，作为三种预训练策略的参照。

3.7 仿真评估

跨八个模拟基准测试不同本体和不同视觉分布下的策略控制。

表 3.7: 仿真评估的实验设置

类别	名称	说明
测试数据	单臂操作	LIBERO、LIBERO-Plus、VLABench；LIBERO-Plus 沿用 LIBERO 模型，不额外微调。
测试数据	双臂操作	RoboTwin2.0-Full、RoboTwin2.0-Clean2Random、RoboDojo。
测试数据	移动操作	RoboCasa365（移动单臂）、EBench（移动双臂）。
测试数据	灵巧手操作	RoboCasa-GR1。
测试方法	统一策略服务协议	每个基准以客户端发送图像、语言和可选本体状态，经相同 Policy Server 返回原生动作。
测试方法	预训练后 SFT	各基准由同一 OpenWAM-α 预训练检查点初始化，并按论文约定分别执行 SFT。
评价指标	仿真任务度量	根据基准记录任务完成率 / 成功率，以及 VLABench、RoboDojo 等任务的相应评分协议。
对比基线	VLA 类模型	OpenVLA、π_0、$\pi_{0.5}$、StarVLA、Xiaomi-Robotics-1、Qwen-RobotManip 等论文列出的适用基线。
对比基线	WAM 类模型	Fast-WAM、LingBot-VA、ABot-M0 / M0.5 等论文列出的适用基线。

3.8 真机评估

通过单臂、双臂和灵巧手三类平台记录策略执行协议，不展示成功数字。

表 3.8: 真机评估的实验设置

类别	名称	说明
测试数据	Franka-Research-3 单臂	六项真实桌面任务；包含堆叠、悬挂和抽屉操作。
测试方法	单臂协议	每项任务 100 条真实示教微调，20 次独立实际测试。
测试数据	RoboDojo-Real 双臂	ARX X5、Piper 与 Piper X；涵盖泛化、精度、长程、记忆和开放式任务。
测试数据	Wuji + Tianji 灵巧手	四项操作任务，覆盖抓取与堆叠、长程操作以及精细操作；该本体未包含在对应预训练混合数据内。
测试方法	策略服务与执行	输入相机、语言和可选本体状态，通过同步推理得到动作块并执行。
评价指标	SR 与 Progress Score	成功率用于任务完成；阶段进度评分用于双臂和灵巧手等多阶段任务。
对比基线	适用 VLA / WAM	论文真机对照中包含 $\pi_{0.5}$、LingBot-VA，并按对应平台的基准表选择其余可用基线。

参考文献

[1] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. Openwam: An open, modular exploration towards systematic world–action model pretraining, 2026.